COMPSCI 689
Lecture 6: Model Complexity and Regularization

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Review

- We now have methods for learning basic models for both regression and classification that are fundamentaly linear functions of both their parameters θ and their inputs $\mathbf{x}$.
- However, real data will typically express more complex non-linear relationships between inputs and outputs.
- To address this problem, we need more complex models as well as effective ways to control complexity.

Basis Function Expansion

- A simple solution to the linearity problem is to apply a set of non-linear functions ϕ_1,...,ϕ_K to the raw feature vector $\mathbf{x}$ to map it in to a new feature space:

$$\phi(\mathbf{x}) = [\phi_1(\mathbf{x}), ..., \phi_K(\mathbf{x})]$$

- This is called a *basis function expansion* since $K > D$ in general. This requires that we know the functions ϕ_1,...,ϕ_K that we want to apply in advance.
- We then define a linear model in this new feature space using a weight vector $\theta \in \mathbb{R}^K$.
- In the regression setting, we obtain the model $\hat{y} = \phi(\mathbf{x})\theta$.
- In the classification setting, we obtain $\hat{y} = \text{sign}(\phi(\mathbf{x})\theta)$.

Basis Function Expansion Examples

- **Univariate Functions:** We can set $\phi_k(\mathbf{x})$ to any univariate function of a single x_d to obtain mappings like $\phi(\mathbf{x}) = [x_1, x_2, \sin(x_1), \exp(x_2)]$, etc.
- **Degree 2 Polynomial Basis:** We include all single features x_d, their squares x_d^2, and all products of two distinct features $x_d x_{d'}$.
- **Degree B Polynomial Basis:** We include all single features x_d, and all unique products of between 2 and B features.
- Don't forget that basis expansion still requires a bias term in the model!
- A key question is how complex should we let the basis expanded model be?

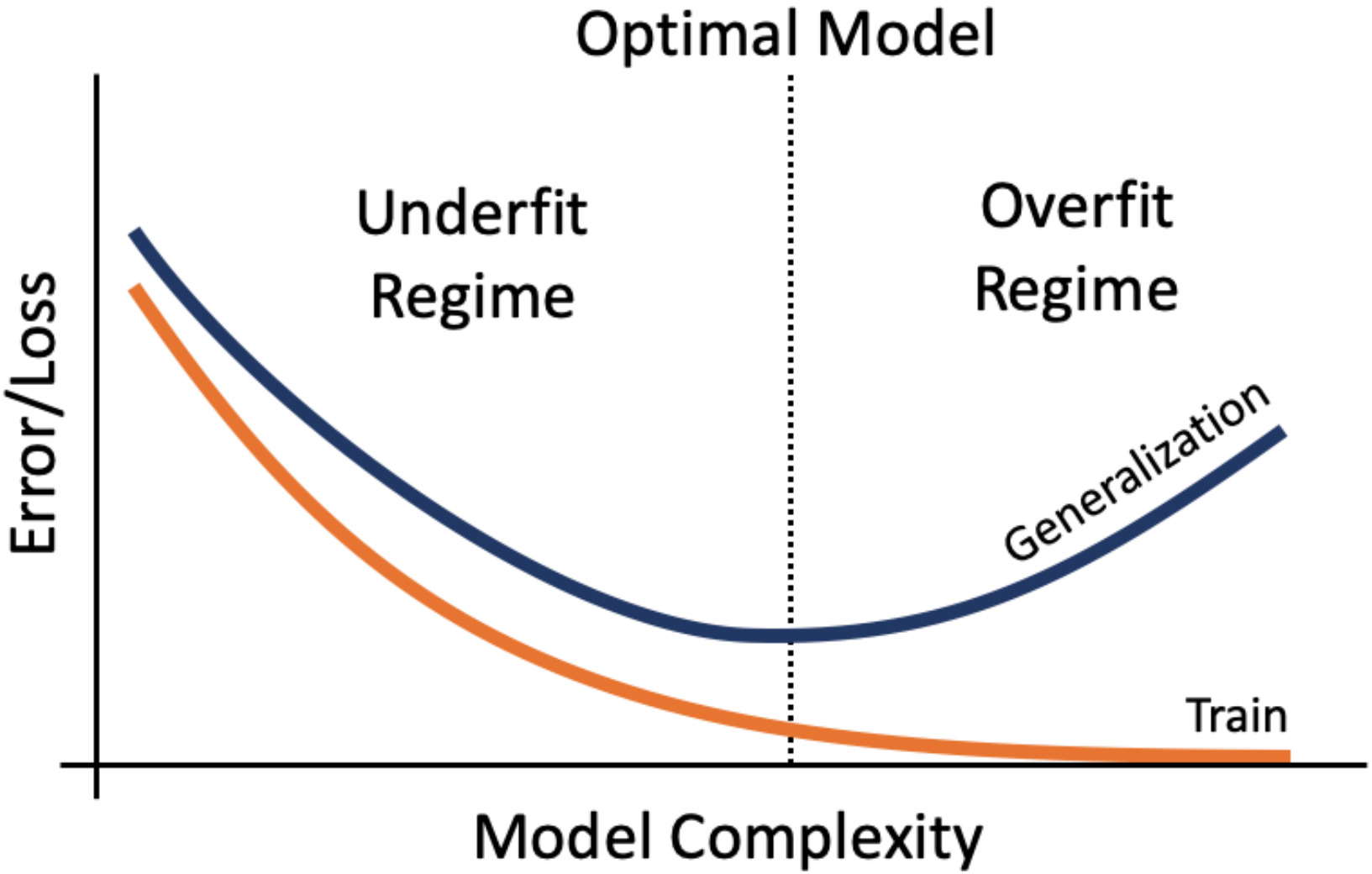
Bias-Variance Trade-Off

- **Bias:** A model is said to have low *bias* if the true relationship between inputs and outputs can be approximated closely by a prediction function expressible by the model.
- **Variance:** A model is said to have low *variance* if the prediction function it constructs is stable with respect to small changes to the training data.
- **Bias-Variance Dilemma:** To achieve low generalization error, we need models that are low-bias and low-variance, but this isn't possible for complex data.

Overfitting and Underfitting

- **Underfitting:** Occurs when the complexity of the model is too low to capture the actual structure in the data, leading to both high training loss and high generalization loss.
- **Overfitting:** Occurs when the complexity of the model is too high so the model is able to closely fit random variations in the data. This results in low training loss and much higher generalization loss.

Overfitting and Underfitting



Controlling Weights

- One way to control the complexity of large linear models is to control the magnitude of their weights.
- In linear models, smaller magnitude weights represent “smoother" functions.

Regularization

- One way to control the magnitude of the weights in a linear model (and many other models) is to add a term to the optimization objective function that penalizes solutions where the weights have large magnitudes.
- The two most common ways to measure the magnitude of the weights are via the ℓ_1 norm $\|\mathbf{w}\|_1$ and squared ℓ_2 norm of the weights $\|\mathbf{w}\|_2^2$.
- These regularization functions can equivalently be thought of as penalizing the distance (under the corresponding norm) of the weights from 0.
- Note that we typically do not regularize the bias term in the model.

Regularized Risk Minimization

- When regularization is applied in the expected risk minimization framework, the resulting framework is referred to as *regularized risk minimization* (RRM).
- Below, we show the RRM lerning problem for the case where the regularization function is based on the squared two norm:

$$\hat{\theta} = \arg\min_{\theta \in \Theta} \left(\left(\sum_{n=1}^{N} L(y_n, f_\theta(\mathbf{x}_n)) \right) + \lambda \sum_{k=1}^{K} w_k^2 \right)$$

- When λ is large, the regularization term encourages the weights to be small and they will eventually be driven to 0.
- When $\lambda = 0$, we recover ERM and the weights are free to take arbitrary values.

Regularized Risk Minimization

- Note that λ serves as a model complexity *hyper-parameter* for the regularization approach to controlling model complexity.
- It is also referred to as the *regularization strength*.
- In the next section, we will discuss how to select model complexity hyper-parameters.

Notes on Regularizers

- In machine learning, the use of a squared ℓ_2 regularizer is often called *weight decay*.
- In the regression setting, using the squared loss and squared ℓ_2 regularizer is referred to as *ridge regression*.
- In the regression setting, using the squared loss and ℓ_1 regularizer is referred to as *the lasso*.
- The use of an ℓ_1 regularizer in a linear model can often be shown to result in weight sparsity.

Hyper-Parameter Selection

- The optimal regularization hyper-parameter value for a given model is the value that leads to the best generalization loss when combined with the model parameter learning procedure.
- Selecting the optimal hyper-parameter is thus part of an extended model learning process that must use data to both select a value for the regularization hyper-parameter and learn the model parameters.
- We can not select regularization hyper-parameters on the data set D_{tr} used to learn the model parameters. Why?
- We can not select regularization hyper-parameters on the test set D_{te} used to evaluate generalization performance. Why?

The Train-Val-Test Experiment

- To select hyper-parameters, learn models, and estimate the generalization performance of the optimal hyper-parameter and model combination, we can use a *train-validation-test* experiment.
- We split the given data into a training set, a *validation* set and a test set.
- We learn the model parameters using the training set for multiple settings of the hyper-parameters.
- We then estimate the generalization loss of the model learned using each hyper-parameter setting on the validation set. This estimate is called the *validation loss*.
- We select the hyper-parameter setting that yields the lowest value of the validation loss.

Performance Estimation

- To get an unbiased estimate of the generalization loss of the model trained using the optimal hyper-parameters, we perform a final evaluation using the test set.
- Once the optimal hyper-parameters have been identified, we can also combine the training set and the validation set and re-fit the model with the optimal hyper-parameters prior to assessing the generalization performance on the test set.

Notes on Selecting Regularization Hyper-Parameters

- It is typical to use an exponential grid of values for λ such as $\lambda \in \{10^{-4}, 10^{-3}, ..., 10^{1}, 10^{2}\}$.
- When running such an experiment with a given range of values for λ, it is important to check that the optimal value over the range is not achieved at the minimum or maximum value tested.
- If it is, the range needs to be extended and the experiment re-run.
- We typically expect a U-shaped curve for validation loss vs regularization strength as the model will be overfit when $\lambda = 0$ and underfit as $\lambda \to \infty$.
- Finally, when comparing the generalization performance of two different models that have complexity hyper-parameters, you must always optimize the hyper-parameters for both models.